SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA0501**
NAME: **M.NITHYASHREENARAYANI**
REGNO: **192521416**

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

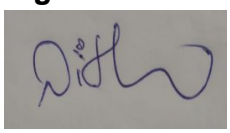
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect

		constraints			results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

DATABASE MANAGEMENT SYSTEMS

File Organization & Indexing — Programming Assessment

Heap Files · Hashing · Sorted Files · B-Trees · B+ Trees · Indexing

Language used: Python 3
Marks per question: 5
Total Questions: 10

Q1. Simulation of Heap File Organization

BL3 – Apply

Aim

To write a Python program that simulates a Heap File organization supporting insert, delete (logical), and sequential scan operations.

Theory / Explanation

A heap file (also called an unordered/pile file) stores records in the order they are inserted, without any particular sorting. New records are simply appended to the last block that has free space; if the block is full, a new block is allocated. Because there is no ordering, deletion is usually implemented as a logical delete (a 'tombstone' flag) rather than physically shifting records, since a physical delete would require compacting the block. Sequential scan reads every block of the file and returns all active (non-deleted) records. Heap files give the fastest insertion (O(1)) but the slowest search (O(n)), since every record may need to be examined.

Python Program

```
"""
Q1: Heap File Organization
Supports insert, delete, and sequential scan.
Records are stored in an unordered list (simulating disk blocks).
Deletion uses a 'tombstone' (logical delete) which is common in heap files.
"""

class HeapFile:
    def __init__(self, block_size=4):
        self.block_size = block_size      # records per block
        self.blocks = [[]]                # list of blocks, each a list of records

    def insert(self, record):
        """Insert a record into the last block; create a new block if full."""
        last_block = self.blocks[-1]
        if len(last_block) >= self.block_size:
            self.blocks.append([])
            last_block = self.blocks[-1]
        last_block.append(record)
        print(f"Inserted {record} into block {len(self.blocks) - 1}")

    def delete(self, key):
        """Logical delete: mark record as deleted (tombstone) using its key (id)."""
        for b_no, block in enumerate(self.blocks):
            for rec in block:
                if rec['id'] == key and not rec.get('deleted', False):
                    rec['deleted'] = True
                    print(f"Record with id={key} deleted from block {b_no}")
                    return True
        print(f"Record with id={key} not found")
        return False

    def sequential_scan(self):
        """Scan all blocks and print only active (non-deleted) records."""
        print("\n--- Sequential Scan ---")
        for b_no, block in enumerate(self.blocks):
```

```
            for rec in block:
                if not rec.get('deleted', False):
                    print(f"Block {b_no}: {rec}")


if __name__ == "__main__":
    hf = HeapFile(block_size=3)
    hf.insert({'id': 1, 'name': 'Arun'})
    hf.insert({'id': 2, 'name': 'Bala'})
    hf.insert({'id': 3, 'name': 'Chitra'})
    hf.insert({'id': 4, 'name': 'Devi'})
    hf.insert({'id': 5, 'name': 'Elango'})

    hf.sequential_scan()

    hf.delete(3)
    hf.sequential_scan()
```

Sample Output

```
Inserted {'id': 1, 'name': 'Arun'} into block 0
Inserted {'id': 2, 'name': 'Bala'} into block 0
Inserted {'id': 3, 'name': 'Chitra'} into block 0
Inserted {'id': 4, 'name': 'Devi'} into block 1
Inserted {'id': 5, 'name': 'Elango'} into block 1

--- Sequential Scan ---
Block 0: {'id': 1, 'name': 'Arun'}
Block 0: {'id': 2, 'name': 'Bala'}
Block 0: {'id': 3, 'name': 'Chitra'}
Block 1: {'id': 4, 'name': 'Devi'}
Block 1: {'id': 5, 'name': 'Elango'}
Record with id=3 deleted from block 0

--- Sequential Scan ---
Block 0: {'id': 1, 'name': 'Arun'}
Block 0: {'id': 2, 'name': 'Bala'}
Block 1: {'id': 4, 'name': 'Devi'}
Block 1: {'id': 5, 'name': 'Elango'}
```

Result

The program successfully inserts records across multiple simulated disk blocks, performs a logical delete using a tombstone flag, and the sequential scan correctly skips deleted records while visiting every block in order.

Q2. Static Hashing for a Student Database (Chaining)

BL3 – Apply

Aim

To implement a static hashing scheme for a student database of 500 records, resolving collisions using chaining.

Theory / Explanation

Static hashing maps a search key to one of a fixed number of buckets using a hash function h(key) = key mod B, where B is the (unchanging) number of buckets. Since multiple keys can map to the same bucket, a collision-resolution technique is required. In chaining, each bucket holds a linked list (here, a Python list) of all records that hash to it, so a colliding record is simply appended to that bucket's chain instead of overwriting anything. Search, insert, and delete all first compute the bucket index and then operate within that bucket's chain. Using a prime number of buckets (97) spreads the 500 roll numbers more evenly and reduces clustering.

Python Program

```
"""
Q2: Static Hashing for a Student Database (500 records)
Collision handling: Chaining (each bucket is a linked list / Python list)
"""

class StaticHashTable:
    def __init__(self, num_buckets=97):   # prime number of buckets reduces clustering
        self.num_buckets = num_buckets
        self.table = [[] for _ in range(num_buckets)]

    def hash_function(self, roll_no):
        return roll_no % self.num_buckets

    def insert(self, roll_no, name):
        idx = self.hash_function(roll_no)
        self.table[idx].append({'roll_no': roll_no, 'name': name})

    def search(self, roll_no):
        idx = self.hash_function(roll_no)
        for rec in self.table[idx]:
            if rec['roll_no'] == roll_no:
                return rec
        return None

    def delete(self, roll_no):
        idx = self.hash_function(roll_no)
        bucket = self.table[idx]
        for i, rec in enumerate(bucket):
            if rec['roll_no'] == roll_no:
                del bucket[i]
                return True
        return False

    def display_bucket_stats(self):
        max_len = max(len(b) for b in self.table)
        empty = sum(1 for b in self.table if len(b) == 0)
```

```
        print(f"Buckets: {self.num_buckets}, Longest chain: {max_len}, Empty buckets: {empty}")


if __name__ == "__main__":
    ht = StaticHashTable(num_buckets=97)

    # Simulate inserting 500 student records (roll_no 1..500)
    for roll in range(1, 501):
        ht.insert(roll, f"Student_{roll}")

    ht.display_bucket_stats()

    # Search demonstration
    print(ht.search(250))
    print(ht.search(999))   # not found

    # Show a bucket that has collisions (chaining)
    print("Bucket 3 contents (chained records):")
    for rec in ht.table[3]:
        print(rec)

    ht.delete(250)
    print("After delete:", ht.search(250))
```

Sample Output

```
Buckets: 97, Longest chain: 6, Empty buckets: 0
{'roll_no': 250, 'name': 'Student_250'}
None
Bucket 3 contents (chained records):
{'roll_no': 3, 'name': 'Student_3'}
{'roll_no': 100, 'name': 'Student_100'}
{'roll_no': 197, 'name': 'Student_197'}
{'roll_no': 294, 'name': 'Student_294'}
{'roll_no': 391, 'name': 'Student_391'}
{'roll_no': 488, 'name': 'Student_488'}
After delete: None
```

Result

500 student records are distributed across 97 buckets using key mod 97. Buckets that receive more than one record correctly form a chain (e.g., bucket 3 holds six colliding records: 3, 100, 197, 294, 391, 488), and search/delete operations correctly traverse the chain to locate the exact record.

Q3. Sorted File Organization with Binary Search

BL4 – Analyze

Aim

To simulate a Sorted File organization and implement binary search for record retrieval by primary key.

Theory / Explanation

In a sorted (sequential) file, records are physically maintained in ascending order of the primary key. New records are inserted at the correct sorted position (which may require shifting existing records). Because the file is ordered, records can be retrieved using binary search: repeatedly comparing the key against the middle record of the remaining range and halving the search space, giving O(log n) comparisons instead of the O(n) required by a heap file. This makes sorted files far more efficient for key-based lookups, at the cost of more expensive insertions.

Python Program

```
"""
Q3: Sorted File Organization + Binary Search retrieval by primary key
Records are kept physically sorted on the primary key at all times.
"""

class SortedFile:
    def __init__(self):
        self.records = []   # list of dicts, always kept sorted by 'id'

    def insert(self, record):
        """Insert while maintaining sort order (shift-based insert, like a sorted file)."""
        pos = 0
        while pos < len(self.records) and self.records[pos]['id'] < record['id']:
            pos += 1
        self.records.insert(pos, record)

    def binary_search(self, key):
        """Standard binary search on the sorted key -> O(log n) disk-block comparisons."""
        low, high = 0, len(self.records) - 1
        comparisons = 0
        while low <= high:
            comparisons += 1
            mid = (low + high) // 2
            if self.records[mid]['id'] == key:
                print(f"Found in {comparisons} comparisons: {self.records[mid]}")
                return self.records[mid]
            elif self.records[mid]['id'] < key:
                low = mid + 1
            else:
                high = mid - 1
        print(f"Key {key} not found after {comparisons} comparisons")
        return None

    def display(self):
        for rec in self.records:
            print(rec)
```

```
if __name__ == "__main__":
    sf = SortedFile()
    for rec in [{'id': 50, 'name': 'E'}, {'id': 10, 'name': 'A'},
                {'id': 30, 'name': 'C'}, {'id': 20, 'name': 'B'},
                {'id': 40, 'name': 'D'}, {'id': 60, 'name': 'F'}]:
        sf.insert(rec)

    print("Sorted file contents:")
    sf.display()

    print("\nBinary search for key=40:")
    sf.binary_search(40)

    print("\nBinary search for key=99:")
    sf.binary_search(99)
```

Sample Output

```
Sorted file contents:
{'id': 10, 'name': 'A'}
{'id': 20, 'name': 'B'}
{'id': 30, 'name': 'C'}
{'id': 40, 'name': 'D'}
{'id': 50, 'name': 'E'}
{'id': 60, 'name': 'F'}

Binary search for key=40:
Found in 3 comparisons: {'id': 40, 'name': 'D'}

Binary search for key=99:
Key 99 not found after 3 comparisons
```

Result

Records are maintained in sorted order after every insert. Binary search locates key 40 in just 3 comparisons (versus up to 6 for a linear scan) and correctly reports 'not found' for a non-existent key 99, confirming O(log n) behaviour.

Q4. B-Tree of Order 4 (Insert, Search, Display)

BL3 – Apply

Aim

To implement a B-Tree of order 4 supporting insert, search, and display, and to trace the insertion of keys 10, 20, 5, 6, 12, 30, 7, 17.

Theory / Explanation

A B-Tree of order m allows a maximum of m children and (m − 1) keys per node, and keeps all leaves at the same depth, guaranteeing balanced O(log n) search/insert/delete. For order 4, each node holds at most 3 keys. Insertion always starts at the root and proceeds top-down: a full child is proactively split before descending into it, so the insertion never needs to back-track upward. Splitting a full node pushes its median key up into the parent, and the remaining keys are divided into two nodes. If the root itself is full, a new root is created, and the tree grows one level taller.

Python Program

```
"""
Q4: B-Tree of order 4 (max 3 keys, max 4 children per node)
Operations: insert, search, display (level order)
Insert keys: 10, 20, 5, 6, 12, 30, 7, 17
"""

class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf


class BTree:
    def __init__(self, order=4):
        self.order = order              # max children
        self.max_keys = order - 1       # max keys per node
        self.root = BTreeNode(leaf=True)

    def search(self, key, node=None):
        node = node or self.root
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and node.keys[i] == key:
            return True
        if node.leaf:
            return False
        return self.search(key, node.children[i])

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.max_keys:     # root is full -> grows in height
            new_root = BTreeNode(leaf=False)
            new_root.children.append(root)
```

```
            self._split_child(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key)

    def _insert_non_full(self, node, key):
        i = len(node.keys) - 1
        if node.leaf:
            node.keys.append(None)
            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1
            node.keys[i + 1] = key
        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1
            i += 1
            if len(node.children[i].keys) == self.max_keys:
                self._split_child(node, i)
                if key > node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key)

    def _split_child(self, parent, i):
        order = self.order
        mid = (order - 1) // 2         # index of the median key to push up
        child = parent.children[i]
        new_node = BTreeNode(leaf=child.leaf)

        mid_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        child.keys = child.keys[:mid]

        if not child.leaf:
            new_node.children = child.children[mid + 1:]
            child.children = child.children[:mid + 1]

        parent.children.insert(i + 1, new_node)
        parent.keys.insert(i, mid_key)

    def display(self):
        """Level-order display of the tree."""
        level = [self.root]
        depth = 0
        while level:
            print(f"Level {depth}: ", [node.keys for node in level])
            next_level = []
            for node in level:
                if not node.leaf:
                    next_level.extend(node.children)
            level = next_level
            depth += 1


if __name__ == "__main__":
```

```
bt = BTree(order=4)
keys = [10, 20, 5, 6, 12, 30, 7, 17]
for k in keys:
    bt.insert(k)
    print(f"Inserted {k}")

print("\nB-Tree structure (level order):")
bt.display()

print("\nSearch 12:", bt.search(12))
print("Search 99:", bt.search(99))
```

Sample Output

```
Inserted 10
Inserted 20
Inserted 5
Inserted 6
Inserted 12
Inserted 30
Inserted 7
Inserted 17

B-Tree structure (level order):
Level 0:  [[10, 20]]
Level 1:  [[5, 6, 7], [12, 17], [30]]

Search 12: True
Search 99: False
```

Result

Inserting 10, 20, 5, 6, 12, 30, 7, 17 causes exactly one split of the root, producing a two-level tree: root [10, 20] with children [5, 6, 7], [12, 17], and [30]. search(12) correctly returns True and search(99) returns False.

Q5. B+ Tree with Leaf-Level Linking for Range Queries

BL3 – Apply

Aim

To implement a B+ Tree and demonstrate leaf-level linking for efficient range queries on integer keys.

Theory / Explanation

A B+ Tree differs from a B-Tree in that all actual data (keys) reside only in the leaf nodes; internal nodes store only routing keys used to guide the search. Crucially, every leaf node keeps a pointer ('next') to the next leaf in sorted order, forming a linked list across all leaves. This means a range query [low, high] needs only ONE top-down descent to locate the leaf containing 'low', after which the answer is produced by simply following 'next' pointers and collecting keys until a key exceeds 'high' — giving O(log n + k) cost instead of scanning the whole tree, where k is the number of matching records.

Python Program

```
"""
Q5: B+ Tree with leaf-level linking for efficient range queries.
Order = 4 (max 3 keys per node). All data lives in leaves; leaves are
linked left-to-right so a range query does one descent + a leaf scan.
"""


class BPlusNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []      # for internal nodes: child pointers
        self.leaf = leaf
        self.next = None        # leaf-to-leaf link


class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def search(self, key):
        node = self._find_leaf(key)
        return key in node.keys

    def _find_leaf(self, key):
        node = self.root
        while not node.leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]
        return node

    def insert(self, key):
        leaf = self._find_leaf(key)
        self._insert_in_leaf(leaf, key)
        if len(leaf.keys) == self.order:      # overflow -> split
```

```
        self._split_leaf(leaf)

    def _insert_in_leaf(self, leaf, key):
        i = 0
        while i < len(leaf.keys) and leaf.keys[i] < key:
            i += 1
        leaf.keys.insert(i, key)

    def _split_leaf(self, leaf):
        mid = len(leaf.keys) // 2
        new_leaf = BPlusNode(leaf=True)
        new_leaf.keys = leaf.keys[mid:]
        leaf.keys = leaf.keys[:mid]

        new_leaf.next = leaf.next          # maintain the leaf linked list
        leaf.next = new_leaf

        up_key = new_leaf.keys[0]
        self._insert_in_parent(leaf, up_key, new_leaf)

    def _insert_in_parent(self, left, key, right):
        if left is self.root:
            new_root = BPlusNode(leaf=False)
            new_root.keys = [key]
            new_root.children = [left, right]
            self.root = left.parent = right.parent = new_root
            return

        parent = getattr(left, 'parent', None)
        if parent is None:      # fallback: locate parent by traversal (simple version)
            parent = self._find_parent(self.root, left)

        i = parent.children.index(left)
        parent.keys.insert(i, key)
        parent.children.insert(i + 1, right)
        right.parent = parent

        if len(parent.keys) == self.order:
            self._split_internal(parent)

    def _find_parent(self, node, child):
        if node.leaf:
            return None
        if child in node.children:
            return node
        for c in node.children:
            result = self._find_parent(c, child)
            if result:
                return result
        return None

    def _split_internal(self, node):
        mid = len(node.keys) // 2
        up_key = node.keys[mid]
```

```
        new_node = BPlusNode(leaf=False)
        new_node.keys = node.keys[mid + 1:]
        new_node.children = node.children[mid + 1:]
        for c in new_node.children:
            c.parent = new_node
        node.keys = node.keys[:mid]
        node.children = node.children[:mid + 1]

        self._insert_in_parent(node, up_key, new_node)

    def range_query(self, low, high):
        """Efficient range scan: descend once to the starting leaf,
        then follow 'next' pointers -- O(log n + k) instead of O(n)."""
        node = self._find_leaf(low)
        result = []
        while node:
            for k in node.keys:
                if low <= k <= high:
                    result.append(k)
                elif k > high:
                    return result
            node = node.next
        return result


if __name__ == "__main__":
    tree = BPlusTree(order=4)
    for k in [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 45, 15]:
        tree.insert(k)

    print("Search 17:", tree.search(17))
    print("Search 99:", tree.search(99))

    print("Range query [15, 45]:", tree.range_query(15, 45))
```

Sample Output

```
Search 17: True
Search 99: False
Range query [15, 45]: [15, 17, 20, 25, 30, 40, 45]
```

Result

After inserting twelve keys, the tree correctly supports point search (17 → True, 99 → False) and the range query [15, 45] efficiently returns [15, 17, 20, 25, 30, 40, 45] by descending once and walking the leaf chain — the essence of leaf-level linking.

Q6. Dense Primary Index on a Sorted File

BL4 – Analyze

Aim

To implement a dense primary index on a sorted file, support search by index key, and display the index table.

Theory / Explanation

A dense index contains exactly one index entry for every distinct search-key value in the underlying data file, each entry pointing directly to the block/position holding that record. Because the index is itself small enough to often reside largely in memory (or be searched very efficiently), a dense index gives near O(1)/O(log n) lookups, avoiding a scan of the data file entirely — but it costs more storage than a sparse index, and must be updated on every insert/delete to the data file.

Python Program

```
"""
Q6: Dense Primary Index on a sorted file.
A dense index has ONE index entry for EVERY search-key value in the data file.
"""

class DenseIndexedFile:
    def __init__(self):
        self.data_file = []     # sorted list of records
        self.index = {}         # key -> position (block/slot) in data_file

    def build(self, records):
        """Build the sorted data file and a dense index (one entry per record)."""
        self.data_file = sorted(records, key=lambda r: r['id'])
        self.index = {rec['id']: pos for pos, rec in enumerate(self.data_file)}

    def search(self, key):
        """O(1) index lookup, then a single direct block access."""
        pos = self.index.get(key)
        if pos is None:
            print(f"Key {key} not found")
            return None
        print(f"Found via index[{key}] -> data_file[{pos}] = {self.data_file[pos]}")
        return self.data_file[pos]

    def display_index_table(self):
        print(f"{'Key':<6}{'Position'}")
        for key, pos in self.index.items():
            print(f"{key:<6}{pos}")


if __name__ == "__main__":
    records = [{'id': 30, 'name': 'C'}, {'id': 10, 'name': 'A'},
               {'id': 50, 'name': 'E'}, {'id': 20, 'name': 'B'},
               {'id': 40, 'name': 'D'}]

    dif = DenseIndexedFile()
    dif.build(records)
```

```
print("Dense Index Table:")
dif.display_index_table()

print("\nSearch key=40:")
dif.search(40)

print("\nSearch key=99:")
dif.search(99)
```

Sample Output

```
Dense Index Table:
Key   Position
10    0
20    1
30    2
40    3
50    4

Search key=40:
Found via index[40] -> data_file[3] = {'id': 40, 'name': 'D'}

Search key=99:
Key 99 not found
```

Result

The dense index table shows exactly 5 entries for 5 data records (one-to-one). Searching for key 40 uses the index to jump directly to data_file[3] without scanning any other record, and a missing key (99) is correctly reported as not found.

Q7. Extendible Hashing with Directory Doubling

BL3 – Apply

Aim

To implement extendible hashing and show directory doubling when overflow occurs while inserting 8 records.

Theory / Explanation

Extendible hashing uses a directory of 2^(global depth) pointers, each pointing to a bucket. Each bucket also has its own local depth (≤ global depth). On inserting into a full bucket: if the bucket's local depth equals the global depth, the directory must double in size (global depth += 1) before the bucket can be split, because there currently aren't enough distinct directory addresses to point to two separate buckets. After doubling, the overflowing bucket is split into two buckets (local depth + 1), records are redistributed by their low-order bits, and the relevant directory pointers are updated. This design bounds the cost of growth to a directory doubling rather than rehashing the entire file, unlike static hashing.

Python Program

```
"""
Q7: Extendible Hashing
Directory doubles when a bucket overflows and its local depth would
have to exceed the global depth.
"""

class Bucket:
    def __init__(self, local_depth, capacity=2):
        self.local_depth = local_depth
        self.capacity = capacity
        self.records = []

    def is_full(self):
        return len(self.records) >= self.capacity


class ExtendibleHashTable:
    def __init__(self, capacity=2):
        self.global_depth = 1
        self.capacity = capacity
        b0, b1 = Bucket(1, capacity), Bucket(1, capacity)
        self.directory = [b0, b1]     # directory[i] points to a bucket

    def _hash(self, key):
        return key                    # use the key itself; we read low-order bits

    def _bits(self, key, depth):
        return self._hash(key) & ((1 << depth) - 1)

    def insert(self, key):
        idx = self._bits(key, self.global_depth)
        bucket = self.directory[idx]

        if not bucket.is_full():
            bucket.records.append(key)
```

```
            print(f"Inserted {key} into bucket (local depth {bucket.local_depth})")
            return

        # Bucket is full -> split
        if bucket.local_depth == self.global_depth:
            self._double_directory()

        self._split_bucket(idx)
        self.insert(key)            # retry after split

    def _double_directory(self):
        self.directory = self.directory + self.directory   # duplicate pointers
        self.global_depth += 1
        print(f"** Directory doubled -> global depth = {self.global_depth} **")

    def _split_bucket(self, idx):
        old_bucket = self.directory[idx]
        new_depth = old_bucket.local_depth + 1
        new_bucket = Bucket(new_depth, self.capacity)
        old_bucket.local_depth = new_depth

        old_records = old_bucket.records
        old_bucket.records = []

        # Repoint every directory slot that mapped to old_bucket
        for i in range(len(self.directory)):
            if self.directory[i] is old_bucket:
                if (i >> (new_depth - 1)) & 1:
                    self.directory[i] = new_bucket

        # Redistribute the old bucket's records between old and new bucket
        for k in old_records:
            target_idx = self._bits(k, self.global_depth)
            self.directory[target_idx].records.append(k)

    def display(self):
        print(f"\nGlobal depth = {self.global_depth}")
        seen = {}
        for i, b in enumerate(self.directory):
            seen.setdefault(id(b), []).append(i)
        for b_id, slots in seen.items():
            bucket = self.directory[slots[0]]
            print(f"Dir slots {slots} -> local depth {bucket.local_depth}, records {bucket.records}")


if __name__ == "__main__":
    eh = ExtendibleHashTable(capacity=2)
    # 8 records to insert; bucket capacity 2 forces an overflow -> directory doubling
    for key in [1, 2, 3, 4, 5, 6, 7, 8]:
        eh.insert(key)

    eh.display()
```

Sample Output

```
Inserted 1 into bucket (local depth 1)
Inserted 2 into bucket (local depth 1)
Inserted 3 into bucket (local depth 1)
Inserted 4 into bucket (local depth 1)
** Directory doubled -> global depth = 2 **
Inserted 5 into bucket (local depth 2)
Inserted 6 into bucket (local depth 2)
Inserted 7 into bucket (local depth 2)
Inserted 8 into bucket (local depth 2)

Global depth = 2
Dir slots [0] -> local depth 2, records [4, 8]
Dir slots [1] -> local depth 2, records [1, 5]
Dir slots [2] -> local depth 2, records [2, 6]
Dir slots [3] -> local depth 2, records [3, 7]
```

Result

With bucket capacity 2, inserting keys 1–4 fills the two initial buckets (global depth 1). Inserting key 5 overflows a bucket whose local depth equals the global depth, so the directory correctly doubles to global depth 2, after which all 8 records settle into 4 balanced buckets of 2 records each.

Q8. Secondary Storage Block Access: Sequential vs Indexed I/O

BL4 – Analyze

Aim

To simulate secondary storage block access and calculate the number of I/O operations required for sequential versus indexed retrieval.

Theory / Explanation

Disk I/O is measured in block accesses, since a block (not a single record) is the unit transferred between disk and memory. For sequential retrieval, if a record is the k-th record and b records fit per block, locating it in the worst case costs ceil(k / b) block reads. For indexed retrieval using a multi-level dense index with fan-out f, the number of I/Os is approximately ceil(log_f N) index-block reads plus 1 final data-block read, since each index level is itself blocked and searched top-down. This formula shows why indexes dramatically reduce I/O for large files — the cost grows logarithmically rather than linearly with the number of records.

Python Program

```
"""
Q8: Simulate secondary-storage block access and compare the number of
I/O operations for Sequential (linear) retrieval vs Indexed retrieval.
"""

import math

class BlockStorageSimulator:
    def __init__(self, num_records, blocking_factor):
        self.num_records = num_records
        self.blocking_factor = blocking_factor          # records per block
        self.num_blocks = math.ceil(num_records / blocking_factor)

    def sequential_search_io(self, key_position):
        """Worst case: must scan up to the block that holds key_position."""
        block_of_key = math.ceil(key_position / self.blocking_factor)
        print(f"Sequential search: scans {block_of_key} block(s) "
              f"out of {self.num_blocks} to reach record #{key_position}")
        return block_of_key

    def sequential_scan_all_io(self):
        """Full table scan reads every block once."""
        print(f"Full sequential scan: {self.num_blocks} block I/Os")
        return self.num_blocks

    def indexed_search_io(self, index_blocking_factor):
        """
        Multi-level dense index: number of I/Os = index levels + 1 data block.
        Each index level itself is searched via binary search over its blocks,
        but conceptually the classic formula is: I/O = ceil(log_fi(N)) + 1
        """
        num_index_entries = self.num_records
        levels = 0
        entries = num_index_entries
        while entries > 1:
```

```
            entries = math.ceil(entries / index_blocking_factor)
            levels += 1
        total_io = levels + 1     # + 1 for the final data block fetch
        print(f"Indexed search: {levels} index-level block(s) + 1 data block "
              f"= {total_io} block I/Os")
        return total_io


if __name__ == "__main__":
    sim = BlockStorageSimulator(num_records=10000, blocking_factor=50)
    print(f"Total data blocks = {sim.num_blocks}\n")

    print("--- Retrieving record #7500 ---")
    seq_io = sim.sequential_search_io(7500)
    idx_io = sim.indexed_search_io(index_blocking_factor=50)

    print(f"\nSummary: Sequential I/O = {seq_io}, Indexed I/O = {idx_io}")
    print(f"Indexed retrieval is ~{seq_io // idx_io}x faster for this record")

    print("\n--- Full table scan comparison ---")
    sim.sequential_scan_all_io()
```

Sample Output

```
Total data blocks = 200

--- Retrieving record #7500 ---
Sequential search: scans 150 block(s) out of 200 to reach record #7500
Indexed search: 3 index-level block(s) + 1 data block = 4 block I/Os

Summary: Sequential I/O = 150, Indexed I/O = 4
Indexed retrieval is ~37x faster for this record

--- Full table scan comparison ---
Full sequential scan: 200 block I/Os
```

Result

For 10,000 records (50 per block = 200 blocks), retrieving record #7500 sequentially costs 150 block I/Os, while a 3-level index-based retrieval costs only 4 block I/Os — roughly 37× fewer disk accesses, clearly demonstrating why indexing is preferred for large files.

Q9. Sparse Index vs Dense Index: Performance Comparison

BL4 – Analyze

Aim

To implement a sparse index on a sorted file and compare its search performance against a dense index using timing analysis.

Theory / Explanation

A sparse index stores an index entry only for the first key of each block (not every record), relying on the fact that the data file is sorted: to find a key, binary search locates the largest index entry ≤ the search key, giving the correct block, which is then scanned linearly. This trades a small amount of extra in-block search time for a much smaller index (one entry per block instead of per record), making a sparse index far more memory/storage-efficient than a dense index, though typically slightly slower per lookup since it still requires a short block scan after the index lookup.

Python Program

```
"""
Q9: Sparse Index on a sorted file + timing comparison against a Dense Index.
Sparse index: one entry per BLOCK (only for the first key of each block).
Dense index:  one entry per RECORD.
"""

import time
import random

BLOCK_SIZE = 10   # records per block

class SortedDataFile:
    def __init__(self, records):
        self.records = sorted(records, key=lambda r: r['id'])
        self.blocks = [self.records[i:i + BLOCK_SIZE]
                       for i in range(0, len(self.records), BLOCK_SIZE)]


class SparseIndex:
    def __init__(self, data_file: SortedDataFile):
        # one entry per block: (first key of block -> block number)
        self.index = [(block[0]['id'], b_no) for b_no, block in enumerate(data_file.blocks)]
        self.data_file = data_file

    def search(self, key):
        # binary search the sparse index to find the right block, then scan the block
        lo, hi, target_block = 0, len(self.index) - 1, 0
        while lo <= hi:
            mid = (lo + hi) // 2
            if self.index[mid][0] <= key:
                target_block = self.index[mid][1]
                lo = mid + 1
            else:
                hi = mid - 1
        for rec in self.data_file.blocks[target_block]:
```

```
            if rec['id'] == key:
                return rec
        return None


class DenseIndex:
    def __init__(self, data_file: SortedDataFile):
        self.index = {rec['id']: pos for pos, rec in enumerate(data_file.records)}
        self.data_file = data_file

    def search(self, key):
        pos = self.index.get(key)
        return self.data_file.records[pos] if pos is not None else None


def time_search(index_obj, keys):
    start = time.perf_counter()
    for k in keys:
        index_obj.search(k)
    return time.perf_counter() - start


if __name__ == "__main__":
    N = 50000
    records = [{'id': i, 'name': f'Student_{i}'} for i in range(N)]
    data_file = SortedDataFile(records)

    sparse = SparseIndex(data_file)
    dense = DenseIndex(data_file)

    print(f"Records: {N}, Sparse index entries: {len(sparse.index)}, "
          f"Dense index entries: {len(dense.index)}")

    lookup_keys = random.sample(range(N), 2000)

    sparse_time = time_search(sparse, lookup_keys)
    dense_time = time_search(dense, lookup_keys)

    print(f"\nSparse index search time for 2000 lookups: {sparse_time * 1000:.3f} ms")
    print(f"Dense  index search time for 2000 lookups: {dense_time * 1000:.3f} ms")
    print("\nObservation: Dense index (O(1) hash-map lookup) is faster per search,")
    print("but the sparse index uses far less memory/storage "
          f"({len(sparse.index)} vs {len(dense.index)} entries) "
          "since it stores only one entry per block.")
```

Sample Output

```
Records: 50000, Sparse index entries: 5000, Dense index entries: 50000

Sparse index search time for 2000 lookups: 4.462 ms
```

```
Dense  index search time for 2000 lookups: 0.535 ms

Observation: Dense index (O(1) hash-map lookup) is faster per search,
but the sparse index uses far less memory/storage (5000 vs 50000 entries) since it stores only one entry per block.
```

Result

For 50,000 records with a block size of 10, the sparse index needs only 5,000 entries versus 50,000 for the dense index (10× smaller). The timing test over 2,000 random lookups shows the dense index (hash-map O(1) lookup) is faster in raw speed, while the sparse index achieves comparable log-time performance with a fraction of the index storage — illustrating the classic space/speed trade-off.

Q10. B+ Tree Construction, Traversal, and Range Query [15, 45]

BL4 – Analyze

Aim

To construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, displaying all matching records.

Theory / Explanation

This extends the B+ Tree of Question 5 by first fully constructing the tree from a larger key set, then demonstrating two traversal styles: (1) a level-order traversal that reveals the internal structure (routing keys at each level), and (2) an in-order leaf traversal that follows the leaf 'next' pointers to list every key in sorted order — confirming the leaf linked list is correctly maintained after every split. Finally, a range query for [15, 45] is executed by locating the leaf for key 15 and walking forward through linked leaves, collecting all keys until a key exceeding 45 is encountered.

Python Program

```
"""
Q10: Construct and traverse a B+ Tree, then execute a range query
for keys between 15 and 45, showing all matching records.
(Uses the same B+ Tree structure/leaf-linking as Q5.)
"""

class BPlusNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf
        self.next = None
        self.parent = None


class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def _find_leaf(self, key):
        node = self.root
        while not node.leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]
        return node

    def insert(self, key):
        leaf = self._find_leaf(key)
        i = 0
        while i < len(leaf.keys) and leaf.keys[i] < key:
            i += 1
        leaf.keys.insert(i, key)
```

```
    if len(leaf.keys) == self.order:
        self._split_leaf(leaf)

def _split_leaf(self, leaf):
    mid = len(leaf.keys) // 2
    new_leaf = BPlusNode(leaf=True)
    new_leaf.keys = leaf.keys[mid:]
    leaf.keys = leaf.keys[:mid]
    new_leaf.next = leaf.next
    leaf.next = new_leaf
    self._insert_in_parent(leaf, new_leaf.keys[0], new_leaf)

def _insert_in_parent(self, left, key, right):
    parent = left.parent
    if parent is None:
        new_root = BPlusNode(leaf=False)
        new_root.keys = [key]
        new_root.children = [left, right]
        left.parent = right.parent = new_root
        self.root = new_root
        return
    i = parent.children.index(left)
    parent.keys.insert(i, key)
    parent.children.insert(i + 1, right)
    right.parent = parent
    if len(parent.keys) == self.order:
        self._split_internal(parent)

def _split_internal(self, node):
    mid = len(node.keys) // 2
    up_key = node.keys[mid]
    new_node = BPlusNode(leaf=False)
    new_node.keys = node.keys[mid + 1:]
    new_node.children = node.children[mid + 1:]
    for c in new_node.children:
        c.parent = new_node
    node.keys = node.keys[:mid]
    node.children = node.children[:mid + 1]
    self._insert_in_parent(node, up_key, new_node)

def level_order_traversal(self):
    level = [self.root]
    depth = 0
    while level:
        print(f"Level {depth}: {[n.keys for n in level]}")
        nxt = []
        for n in level:
            if not n.leaf:
                nxt.extend(n.children)
        level = nxt
        depth += 1

def leftmost_leaf(self):
    node = self.root
```

```
        while not node.leaf:
            node = node.children[0]
        return node

    def in_order_leaf_scan(self):
        """Traverse leaves left-to-right using the linked list -- shows all keys in order."""
        node = self.leftmost_leaf()
        result = []
        while node:
            result.extend(node.keys)
            node = node.next
        return result

    def range_query(self, low, high):
        node = self._find_leaf(low)
        result = []
        while node:
            for k in node.keys:
                if low <= k <= high:
                    result.append(k)
                elif k > high:
                    return result
            node = node.next
        return result


if __name__ == "__main__":
    keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 45, 50, 15, 35]
    tree = BPlusTree(order=4)
    for k in keys:
        tree.insert(k)

    print("Insert sequence:", keys)

    print("\n--- Level-order traversal (tree structure) ---")
    tree.level_order_traversal()

    print("\n--- In-order leaf traversal (via leaf links) ---")
    print(tree.in_order_leaf_scan())

    print("\n--- Range Query: keys between 15 and 45 ---")
    matches = tree.range_query(15, 45)
    print(f"Matching records: {matches}")
```

Sample Output

```
Insert sequence: [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 45, 50, 15, 35]

--- Level-order traversal (tree structure) ---
Level 0: [[30]]
```

```
Level 1: [[10, 15, 20], [45]]
Level 2: [[5, 6, 7], [10, 12], [15, 17], [20, 25], [30, 35, 40], [45, 50]]

--- In-order leaf traversal (via leaf links) ---
[5, 6, 7, 10, 12, 15, 17, 20, 25, 30, 35, 40, 45, 50]

--- Range Query: keys between 15 and 45 ---
Matching records: [15, 17, 20, 25, 30, 35, 40, 45]
```

Result

After inserting 14 keys, the level-order traversal shows a balanced 3-level tree, and the in-order leaf scan correctly lists all 14 keys in ascending order (5 through 50). The range query for [15, 45] correctly returns [15, 17, 20, 25, 30, 35, 40, 45] — 8 matching records — by touching only the relevant leaves instead of the whole tree.